CS3.301: Operating Systems and Networks
Lecture 20: Condition Variables

INSTRUCTOR: DR. DEEPAK GANGADHARAN

Roadmap for today

1

Why locks are not enough

Waiting for a condition; parent waits for child (join)

2

Condition variables

wait() and signal(); state variable + lock + CV

3

Producer / consumer

From broken to correct; Mesa semantics; two CVs

4

Covering conditions

Who should be woken? broadcast()

Locks are not enough

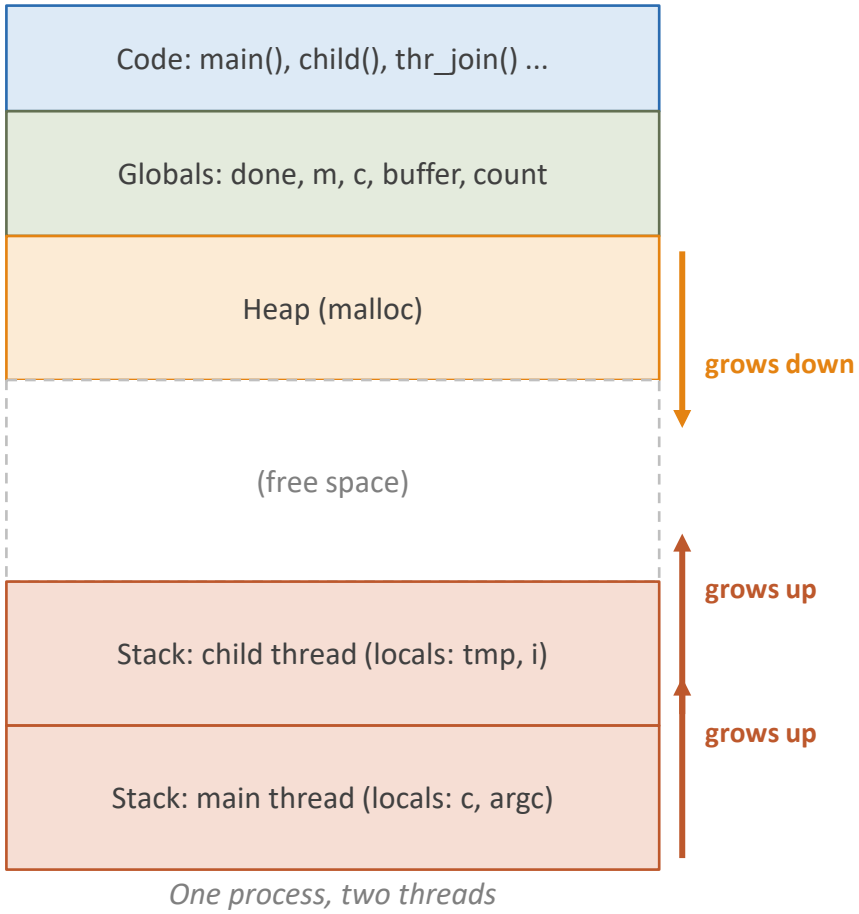
- Locks give **mutual exclusion**: one thread at a time in a critical section
- Often a thread must instead **wait until a condition is true**
- Example: a parent must wait for its child to finish, i.e. `join()`
- Required output, whatever the scheduling order:
  - parent: begin → child → parent: end
- Question: how should the parent wait?

```
void *child(void *arg) {
    printf("child\n");
    // XXX how to indicate we are done?
    return NULL;
}
int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(&c, NULL, child, NULL);
    // XXX how to wait for child?
    printf("parent: end\n");
    return 0;
}
```

Desired output

```
parent: begin child parent: end
```

Threads share one address space



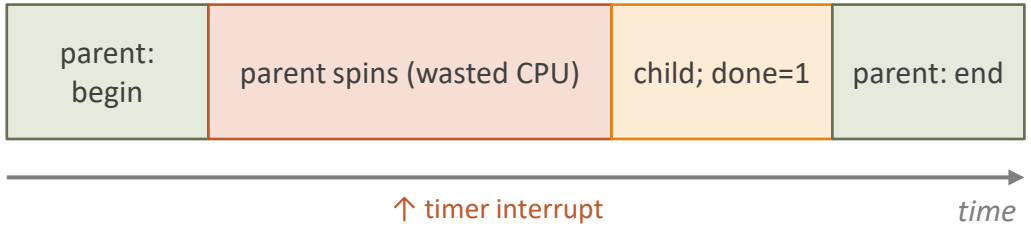
One process, two threads

- Shared by all threads: **code, globals and heap**
- Private to each thread: its **own stack** (locals, return addresses), registers and PC
- Threads communicate through shared variables such as `done`, `buffer`, `count`
- Shared state needs two kinds of coordination:
  - a **lock** so updates do not interleave
  - a **condition variable** so a thread can sleep until the state is what it needs

Attempt 1: spin on a shared variable

```
volatile int done = 0;
void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}
int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(&c, NULL, child, NULL);
    while (done == 0)
        ; // spin
    printf("parent: end\n");
    return 0;
}
```

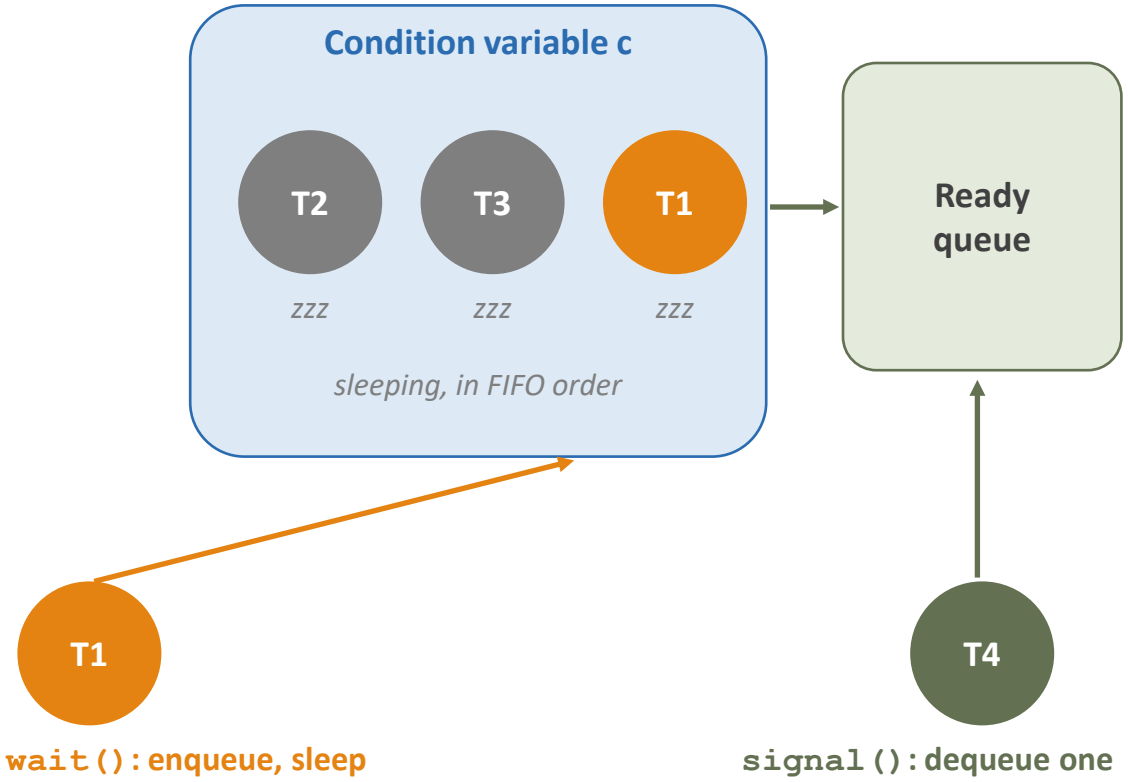
One CPU, parent scheduled first



The crux: how to wait for a condition?
Spinning works but wastes CPU cycles, and in some cases is incorrect.
We want the parent to **sleep** until the condition becomes true.

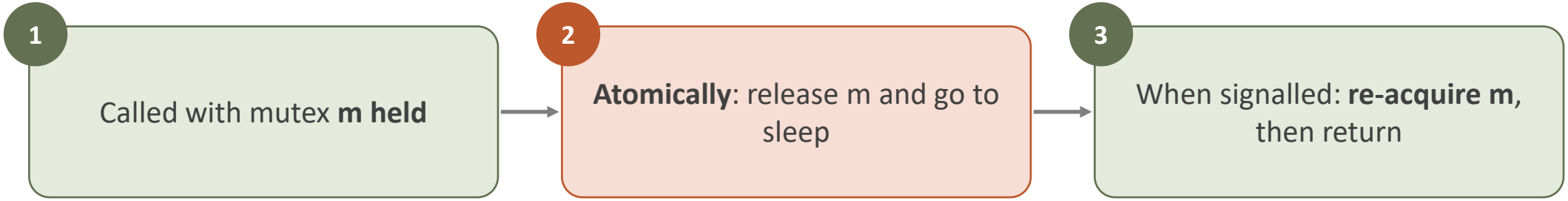
A condition variable is a queue of waiters

- A **condition variable (CV)**: an explicit queue of threads waiting for some state to become true
- `wait()`: join the queue and **sleep**
- `signal()`: after changing the state, **wake** a waiting thread
- POSIX: `pthread_cond_t c;` (must be initialised)



The API: wait() and signal()

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);    // wait(c, m)
pthread_cond_signal(pthread_cond_t *c);                      // signal(c)
```



What `wait(c, m)` does

- Atomic release-and-sleep prevents a signal slipping in between the two steps (a **lost wakeup**)
- `signal()` wakes one waiter; **if nobody is waiting, the signal is lost**, not remembered

Join with a condition variable

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  c = PTHREAD_COND_INITIALIZER;

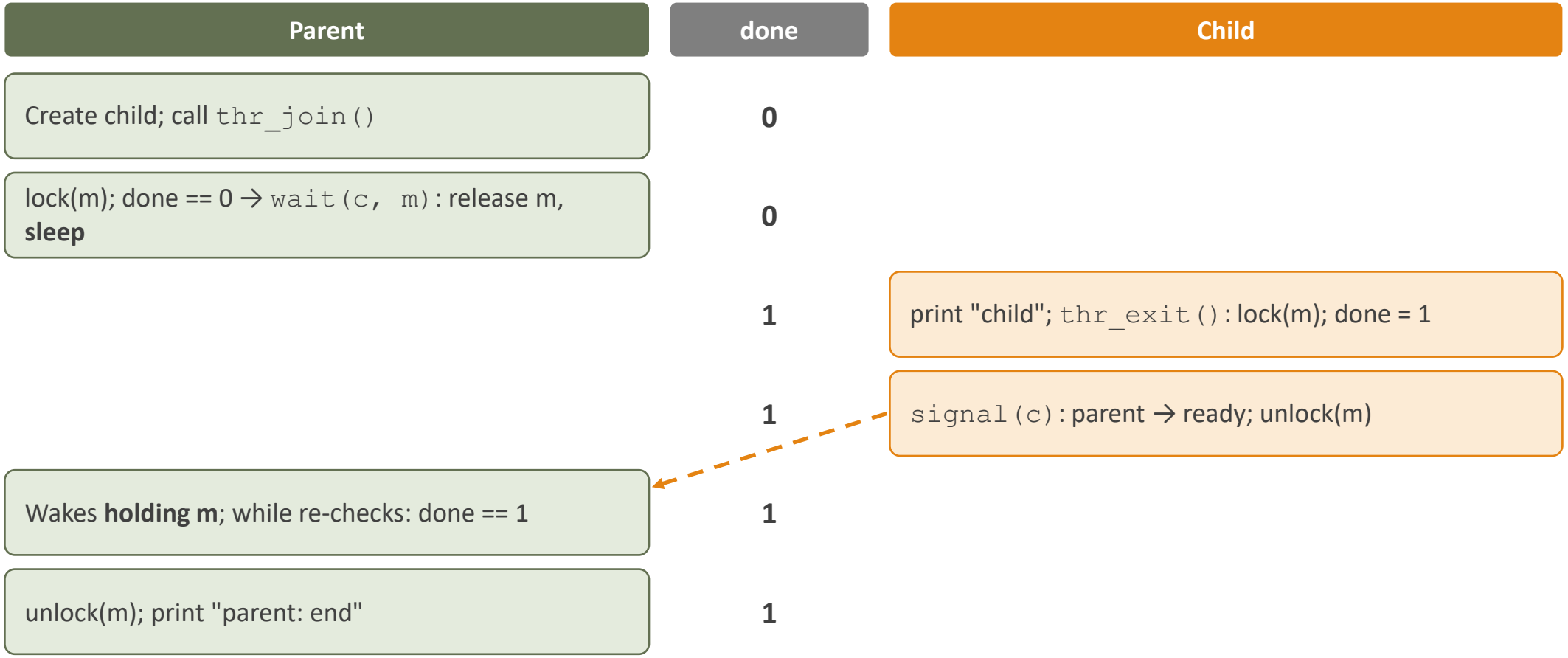
void thr_exit() {           // called by child
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}
void thr_join() {           // called by parent
    Pthread_mutex_lock(&m);
    while (done == 0)
        Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}
 // child: printf("child"); thr_exit();
 // main:  create child; thr_join(); print end
```

State variable: done
Records what happened: has the child finished?

Mutex: m
Protects `done`; held around every wait and signal

Condition variable: c
Where the parent sleeps while `done == 0`

Trace 1: the parent waits first



Trace 2: the child runs first

Parent	done	Child
Create child; parent is descheduled	**0**	
	1	print "child"; `thr_exit()`: lock(m); done = 1
	1	`signal(c)`: **nobody waiting**, nothing happens; unlock(m)
`thr_join()`: lock(m); done == 1 → **no wait**	**1**	
unlock(m); print "parent: end"	**1**	

The signal was lost, but `done` **remembered the event**: the parent checks the state, not the signal

Broken 1: no state variable

```
void thr_exit() {
    Pthread_mutex_lock(&m);
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

void thr_join() {
    Pthread_mutex_lock(&m);
    Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}
```

Child runs first:

Child	lock(m); `signal(c)`: **no waiter, signal lost**; unlock(m)
Parent	lock(m); `wait(c, m)` → sleeps
Parent	**Nobody will ever signal again: sleeps forever**

The state variable records what happened; sleeping, waking and locking are built around it

Broken 2: no lock

```
void thr_exit() {
    done = 1;
    Pthread_cond_signal(&c);
}

void thr_join() {
    if (done == 0)
        Pthread_cond_wait(&c);
}
```

Step	Parent	Child	done
1	`if (done == 0)`: true		0
2	(interrupted before wait)		0
3		done = 1	1
4		`signal(c)`: no waiter	1
5	**wait(c): sleeps forever**		1

Check-then-sleep must be atomic
Holding m stops the child from changing `done` between the check and the sleep; `wait()` releases m only once the thread is asleep

Rules for using condition variables

1 **Use a state variable**

The CV is only a place to sleep. The state (`done`, `count`) records whether the condition holds.

2 **Hold the lock for wait and signal**

Required for `wait()`; simplest and safe for `signal()`.

3 **Re-check in a while loop**

After waking, test the condition again before proceeding.

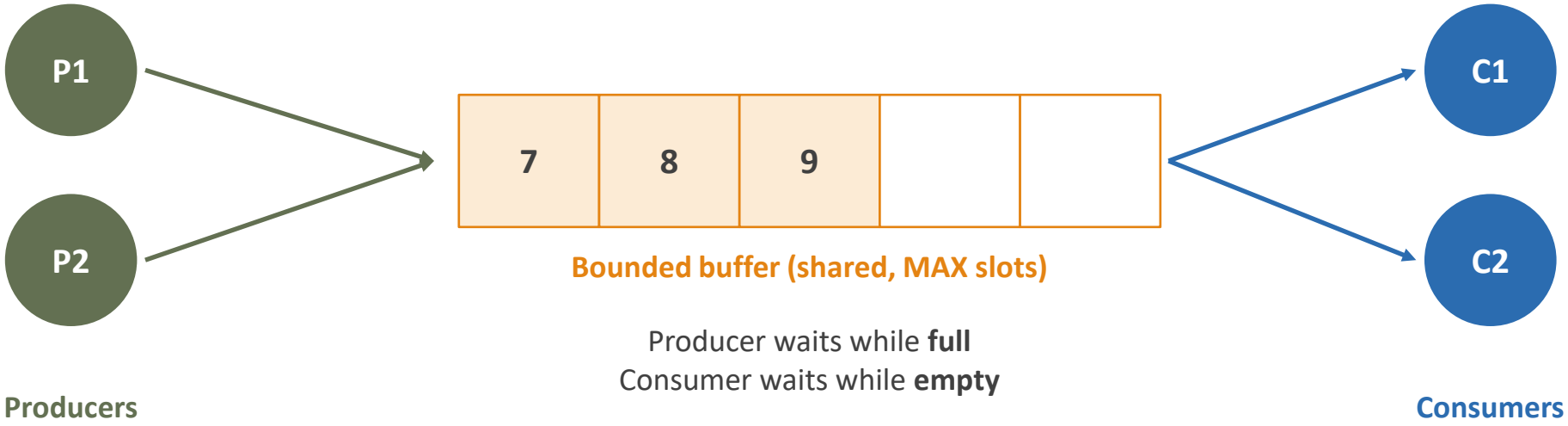
```
lock(&m);
while (!condition)
    wait(&cv, &m);
/* ... use the state ... */
unlock(&m);
```

Waiting thread

```
lock(&m);
/* change the state */
signal(&cv);
unlock(&m);
```

Signalling thread

The producer/consumer problem



Multi-threaded web server
Producer puts HTTP requests in a work queue; worker threads consume them

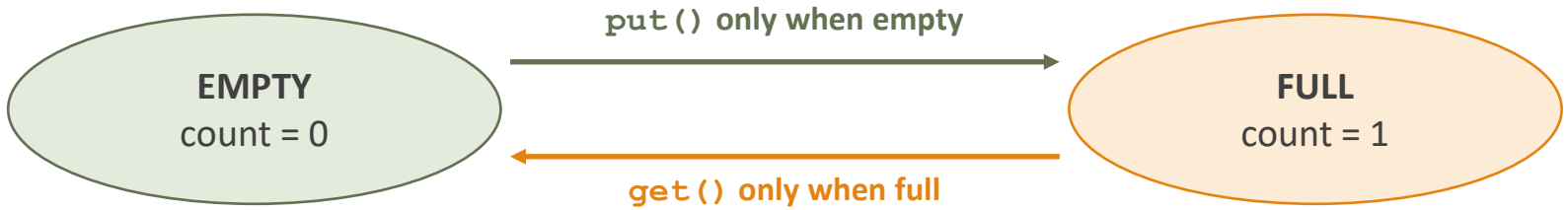
Unix pipe
`grep foo file.txt | wc -l`: grep produces lines, wc consumes them via a kernel buffer

Start simple: a one-slot buffer

```
int buffer;
int count = 0;   // initially empty

void put(int value) {
    assert(count == 0);
    count = 1;
    buffer = value;
}
int get() {
    assert(count == 1);
    count = 0;
    return buffer;
}
```

```
void *producer(void *arg) {
    int loops = (int) arg;
    for (int i = 0; i < loops; i++)
        put(i);
}
void *consumer(void *arg) {
    while (1) {
        int tmp = get();
        printf("%d\n", tmp);
    }
}
```



Broken solution v1: one CV and if

```
void *producer(void *arg) {
  for (int i = 0; i < loops; i++) {
    Pthread_mutex_lock(&mutex);           // p1
    if (count == 1)                       // p2
      Pthread_cond_wait(&cond, &mutex);   // p3
    put(i);                               // p4
    Pthread_cond_signal(&cond);           // p5
    Pthread_mutex_unlock(&mutex);         // p6
  }
}
void *consumer(void *arg) {
  for (int i = 0; i < loops; i++) {
    Pthread_mutex_lock(&mutex);           // c1
    if (count == 0)                       // c2
      Pthread_cond_wait(&cond, &mutex);   // c3
    int tmp = get();                      // c4
    Pthread_cond_signal(&cond);           // c5
    Pthread_mutex_unlock(&mutex);         // c6
    printf("%d\n", tmp);
  }
}
```

Shared: `cond_t cond; mutex_t mutex;` *plus the one-slot* `buffer` *and* `count`

- One mutex, **one** condition variable
- Producer waits while the buffer is full (p1–p3)
- Consumer waits while it is empty (c1–c3)
- Works with **1 producer + 1 consumer**
- With **2 consumers** it has two bugs:
  - **if instead of while**
  - **one CV for both kinds of waiter**

Trace v1: a second consumer sneaks in

Tc1 (consumer 1)	Tc2 (consumer 2)	Tp (producer)	count	Comment
c1–c3 · Sleep	Ready	Ready	0	Nothing to get
Sleep	Ready	p1, p2, p4 · Run	1	Buffer now full
Ready	Ready	p5 · Run	1	Tc1 awoken (but not running)
Ready	Ready	p6, p1, p2 · Run	1	
Ready	Ready	p3 · Sleep	1	Buffer full; producer sleeps
Ready	c1, c2 · Run	Sleep	1	Tc2 sneaks in...
Ready	c4 · Run	Sleep	0	...and grabs the data
Ready	c5, c6 · Run	Ready	0	Tp awoken
c4 · Run	Ready	Ready	0	Oh oh! No data: assert fails

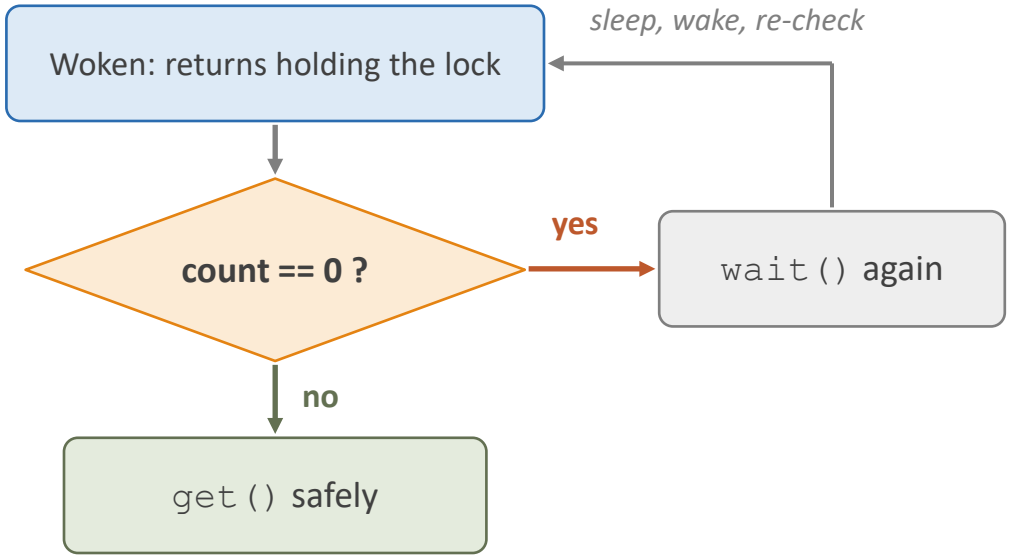
Run Ready Sleep

Tc1 was woken when the buffer was full, but it was **empty by the time Tc1 ran**

Fix 1: use while, not if

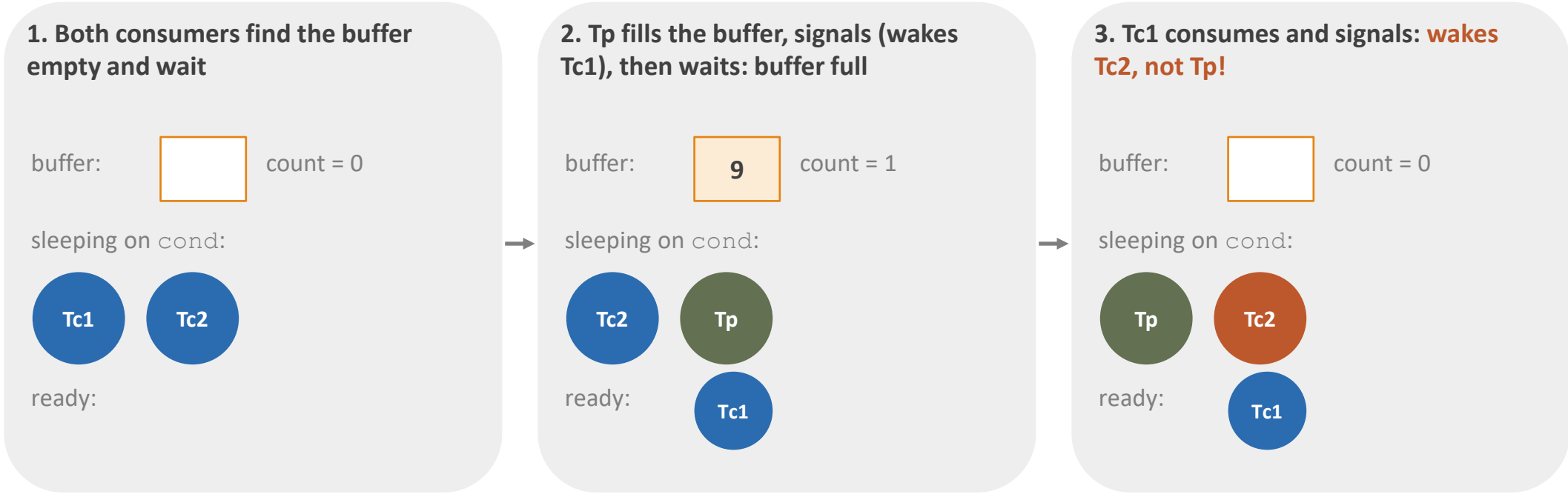
```
 // consumer, v1 (broken)
Pthread_mutex_lock(&mutex);           // c1
if (count == 0)                       // c2
  Pthread_cond_wait(&cond, &mutex);   // c3
int tmp = get();                      // c4
```

```
 // consumer, v2
Pthread_mutex_lock(&mutex);           // c1
while (count == 0)                    // c2
  Pthread_cond_wait(&cond, &mutex);   // c3
int tmp = get();                      // c4
```



- Woken thread re-checks `count` before `get()`
- If another consumer took the item, it **sleeps again**
- Same change in the producer (p2)
- **Still broken** with one CV: see next slide

Broken solution v2: waking the wrong thread



Tc2 re-checks, finds it empty and sleeps; Tc1 also sleeps. **All three threads asleep.** Signals must be **directed**.

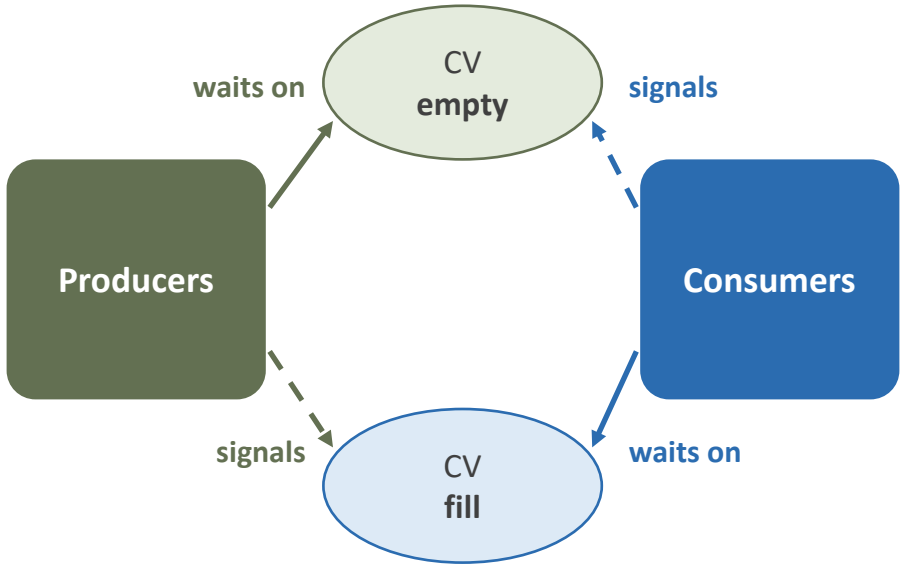
Trace v2: everyone falls asleep

Tc1 (consumer 1)	Tc2 (consumer 2)	Tp (producer)	count	Comment
c1–c3 · Sleep	Ready	Ready	0	Nothing to get
Sleep	c1–c3 · Sleep	Ready	0	Nothing to get
Sleep	Sleep	p1, p2, p4 · Run	1	Buffer now full
Ready	Sleep	p5 · Run	1	Tc1 awoken
Ready	Sleep	p6, p1, p2 · Run	1	
Ready	Sleep	p3 · Sleep	1	Must sleep (full)
c2 · Run	Sleep	Sleep	1	Recheck condition
c4 · Run	Sleep	Sleep	0	Tc1 grabs data
c5 · Run	Ready	Sleep	0	Oops! Woke Tc2
c6, c1, c2 · Run	Ready	Sleep	0	
c3 · Sleep	Ready	Sleep	0	Nothing to get
Sleep	c2 · Run	Sleep	0	Recheck: still empty
Sleep	c3 · Sleep	Sleep	0	Everyone asleep...

Run Ready Sleep

Fix 2: two condition variables

```
cond_t empty, fill;  mutex_t mutex;
void *producer(void *arg) {
  for (int i = 0; i < loops; i++) {
    Pthread_mutex_lock(&mutex);
    while (count == 1)
      Pthread_cond_wait(&empty, &mutex);
    put(i);
    Pthread_cond_signal(&fill);
    Pthread_mutex_unlock(&mutex);
  }
}
void *consumer(void *arg) {
  for (int i = 0; i < loops; i++) {
    Pthread_mutex_lock(&mutex);
    while (count == 0)
      Pthread_cond_wait(&fill, &mutex);
    int tmp = get();
    Pthread_cond_signal(&empty);
    Pthread_mutex_unlock(&mutex);
    printf("%d\n", tmp);
  }
}
```



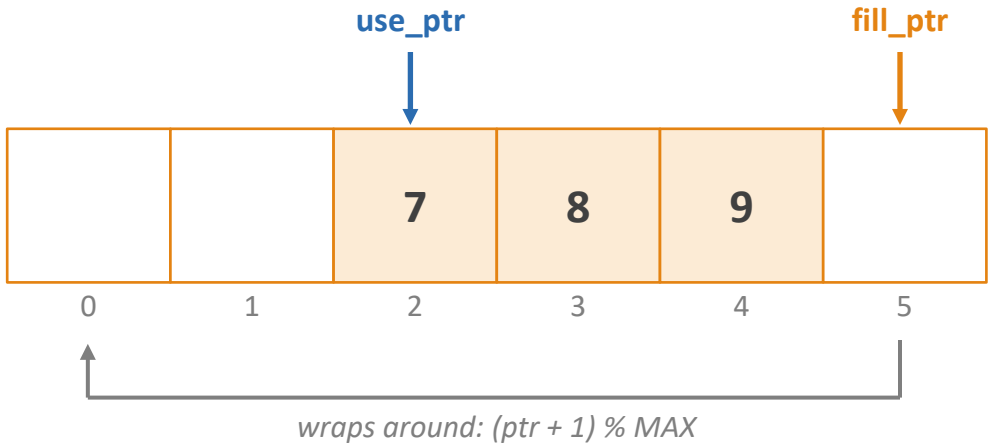
A consumer can only wake producers, and a producer can only wake consumers

General solution: a buffer with MAX slots

```
int buffer[MAX];
int fill_ptr = 0;
int use_ptr  = 0;
int count    = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

Example: MAX = 6, count = 3



- `put()` writes at `fill_ptr`; `get()` reads at `use_ptr`
- `count` = number of full slots, 0 … MAX

The correct producer/consumer solution

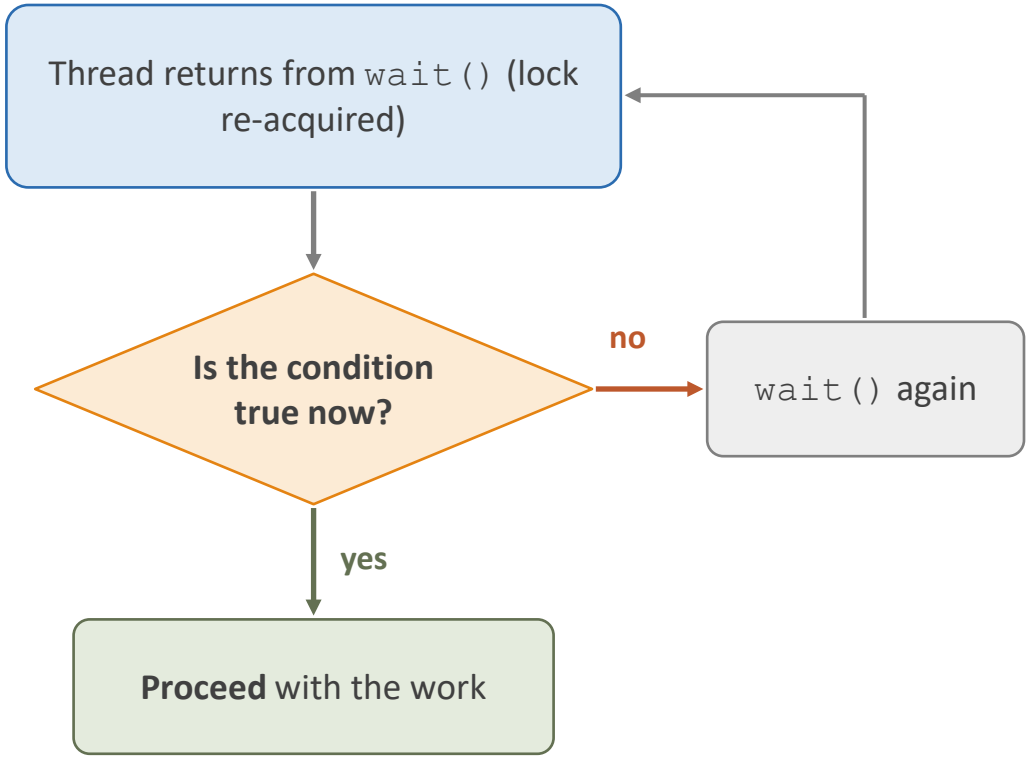
```
void *producer(void *arg) {
  for (int i = 0; i < loops; i++) {
    Pthread_mutex_lock(&mutex);           // p1
    while (count == MAX)                  // p2
      Pthread_cond_wait(&empty, &mutex);  // p3
    put(i);                               // p4
    Pthread_cond_signal(&fill);           // p5
    Pthread_mutex_unlock(&mutex);         // p6
  }
}
void *consumer(void *arg) {
  for (int i = 0; i < loops; i++) {
    Pthread_mutex_lock(&mutex);           // c1
    while (count == 0)                    // c2
      Pthread_cond_wait(&fill, &mutex);   // c3
    int tmp = get();                      // c4
    Pthread_cond_signal(&empty);          // c5
    Pthread_mutex_unlock(&mutex);         // c6
    printf("%d\n", tmp);
  }
}
```

Run with MAX = 2, one producer, one consumer

#	Event	count
1	Tp: put(0), signal(fill)	1
2	Tp: put(1), signal(fill)	2
3	Tp: count == MAX → wait(empty)	2
4	Tc: get() → 0, signal(empty): Tp ready	1
5	Tp: re-check, count < MAX → put(2)	2
6	Tc: get() → 1, 2; count == 0 → wait(fill)	0

*Producer sleeps only when **all** slots are full; consumer only when **all** are empty*

Tip: always use while, not if



1. **Mesa semantics**
   A signal only makes the waiter ready; others may run first and change the state

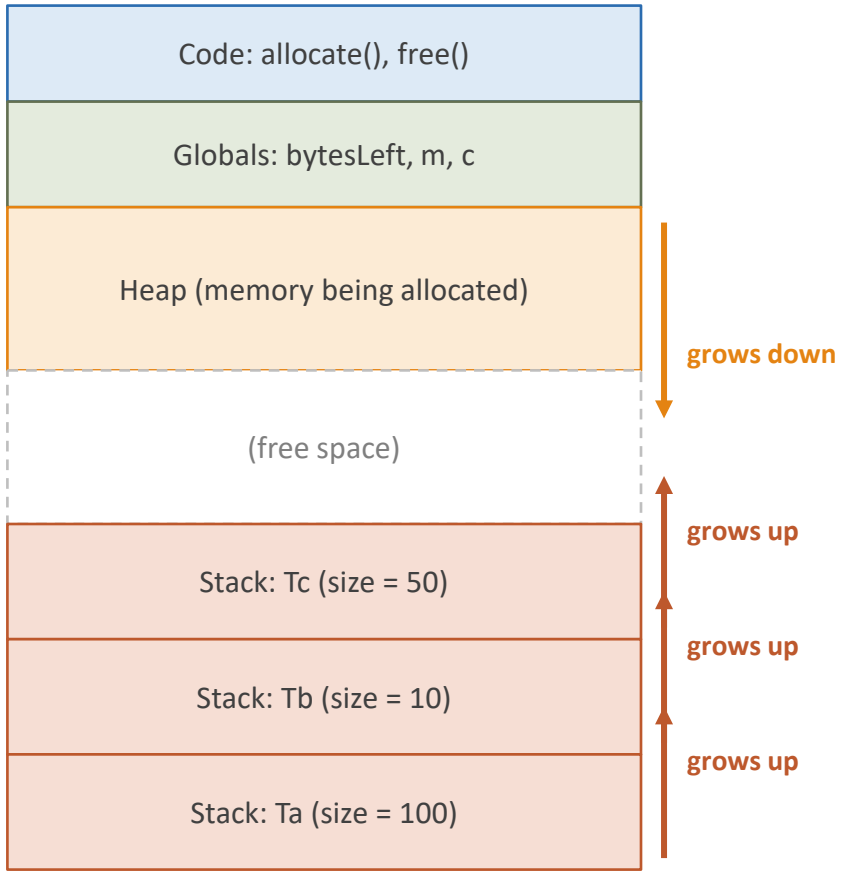
2. **A thread sneaks in**
   Another consumer can take the item before the woken thread runs (trace v1)

3. **Spurious wakeups**
   Some implementations wake two threads for one signal, or wake with no signal

Covering conditions: a memory allocator

```
int bytesLeft = MAX_HEAP_SIZE;  // free bytes
cond_t c;  mutex_t m;

void *allocate(int size) {
    Pthread_mutex_lock(&m);
    while (bytesLeft < size)
        Pthread_cond_wait(&c, &m);
    void *ptr = ...;  // get mem from heap
    bytesLeft -= size;
    Pthread_mutex_unlock(&m);
    return ptr;
}
void free(void *ptr, int size) {
    Pthread_mutex_lock(&m);
    bytesLeft += size;
    Pthread_cond_signal(&c);  // whom to signal??
    Pthread_mutex_unlock(&m);
}
```



Which waiter should free() wake?

Ta `allocate(100)`: 0 < 100 → wait

Tb `allocate(10)`: 0 < 10 → wait

Tc `free(50)`: bytesLeft = 50; `signal(c)`

Ta Woken: 50 < 100 → **waits again**

Tb needs only 10 bytes but stays asleep

Heap after free(50)



CV queue after step 4

Tb **Ta** *Both asleep, though Tb could run*

Fix: broadcast as a covering condition

```
Pthread_cond_broadcast(&c);   // in free(): wake ALL waiters
```

	signal()	broadcast()
Wakes	One waiting thread	**All** waiting threads
Correct here?	**No: may wake Ta instead of Tb**	**Yes: Tb proceeds, Ta sleeps again**
Cost	Minimal	Extra wakeups, lock contention, context switches
Use when	You know which waiter can proceed (e.g. separate CVs)	You cannot tell which waiter can proceed

If code only works with broadcast and you don't see why it should need it, **you probably have a bug**

Summary

1. **Sleep, don't spin**

   CVs let a thread sleep until another thread changes the state it needs

2. **The recipe**

   State variable + lock + CV, and re-check the condition in a `while` loop

3. **Signals are hints**

   Mesa semantics: the state may change before the woken thread runs

4. **Direct your signals**

   Separate CVs (`empty`/`fill`); `broadcast()` for covering conditions

Thank You!!